AI Frameworks

-> AI frameworks are toolkits with built-in AI functions that help developers build models without coding everything from scratch.

->Their source code may be available on GitHub, AI models built with them may be shared on Hugging Face, and those models may be run locally using Ollama.

->However, GitHub, Hugging Face, and Ollama are platforms/tools, not AI frameworks themselves.

What Does an AI Framework Do?

->It provides ready-made functions for:

- Building neural networks.
- Training AI models.
- Processing data.
- Running predictions (inference).
- Saving and loading models.

Example: Suppose you want to build an AI that recognizes cats and dogs.

Without an AI framework:

->You must code all the neural network mathematics yourself.

With an AI framework like TensorFlow:

->You use built-in functions to create the model and train it with images.

Cat & Dog Images
↓
TensorFlow
↓
Train Neural Network
↓
AI Model
↓
Predict: Cat or Dog

Tensor Flow

-> TensorFlow is an AI framework developed by Google that provides ready-made functions for building and training AI models.

->Instead of writing all the deep learning mathematics from scratch, developers use TensorFlow's built-in tools to create AI systems more easily.

Why is TensorFlow Needed?

->Suppose you want to create an AI model that can recognize cats and dogs.

Without TensorFlow:

- You must write all the neural network equations.
- You must code the training process.
- You must handle the mathematical calculations yourself.

With TensorFlow:

- You use built-in functions.
- TensorFlow handles most of the complex math.
- You focus on designing the AI model.

How Does TensorFlow Work?

Training Data
↓
Developer writes Python code
↓
TensorFlow provides built-in AI functions
↓
Model is trained
↓
Trained AI Model
↓
Used for predictions

Example: Suppose you want to create an AI that detects whether an email is spam.

Email Data
↓
TensorFlow
↓
Train AI Model
↓
Trained Spam Detector
↓
New Email Arrives
↓
AI Predicts: Spam or Not Spam

PyTorch

-> TensorFlow is Google's AI toolkit and PyTorch is Meta's AI toolkit. Both perform similar functions and provide ready-made AI libraries and functions, so developers do not have to write all the deep learning and neural network code from scratch. They are used to build and train AI models.

Example

Suppose you want to create an AI that recognizes cats and dogs.

- Without TensorFlow/PyTorch → You write all the mathematical equations and training algorithms yourself.
- With TensorFlow/PyTorch → You use their built-in functions to create and train the model much more easily.

->AI research and experimentation means testing and improving new AI ideas and models.

->Production and deployment mean taking the final trained AI model and making it available for real users to use in real-world applications.

Example: Imagine building an AI chatbot

Research & Experimentation
↓
Try different AI architectures and improve accuracy.

Deployment
↓
Upload the final chatbot model to a cloud server.

Production
↓
Millions of users chat with the AI every day.

Scikit-Learn

-> Scikit-Learn is mainly used for traditional machine learning problems where developers train models using input data and built-in ML algorithms. It is suitable for simpler prediction and classification tasks.

->For complex AI systems like LLMs that require deep learning and neural networks, frameworks such as TensorFlow and PyTorch are generally used instead.

What is "Traditional ML"?

Traditional Machine Learning uses algorithms like:

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forests
- Support Vector Machines (SVM)
- K-Means Clustering

->These algorithms **do not use deep neural networks with many layers** like LLMs or image generation models do.

Can Scikit-Learn be used for a Language Model?

🟡 **Technically, yes, but only for very simple NLP (Natural Language Processing) tasks.**

For example, you could use Scikit-Learn to build:

- Spam email detection.
- Sentiment analysis (Positive/Negative).
- Document classification.

But you **cannot realistically build a modern LLM like ChatGPT or Llama using only Scikit-Learn** because those require deep learning and huge neural networks.

Open CV

-> OpenCV (Open Source Computer Vision Library) is an open-source framework used for image processing and computer vision applications.

What Can OpenCV Do?

✅ Read and display images.
✅ Process videos.
✅ Detect faces and objects.
✅ Recognize shapes and colors.
✅ Track moving objects.
✅ Work with cameras in real time.

Example

Suppose you want to build a **face unlock system** for a phone.

Phone Camera
↓
OpenCV
↓

Detects and locates the face
↓
AI Model compares with stored face data
↓
Phone Unlocks

Here:

- **OpenCV** helps detect and process the image.
- **The AI model** decides whose face it is.

AI Data Infrastructure

Vector database:

-> A vector database does not store the actual intelligence of the AI or the model parameters. It stores embeddings, which are numerical representations of the meaning of data.
->When RAG is used, the user's question is converted into an embedding and compared with the stored embeddings in the vector database.
->The most relevant information is retrieved and passed to the LLM, which then generates the final answer by predicting the next tokens.

Example (ChatGPT-like RAG system)

Suppose there are three documents already processed and stored:

Document	Embedding (simplified)
AI Policy PDF	[0.81, -0.12, 0.45]
Leave Policy PDF	[0.15, 0.72, -0.30]
Internship Guidelines	[0.68, -0.08, 0.50]

Now you ask:

"Tell me about the company's AI policy."

The system works like this:

Your Question
↓
Tokenizer
↓

Tokens
↓
Embedding Model
(Question converted into a vector)
↓
Vector Database
(Finds the most similar stored embedding)
↓
RAG retrieves the AI Policy document
↓
LLM reads the retrieved information
↓
Neural Network predicts next tokens-> Final answer to the user

Embedding

-> An embedding is the process of converting tokens (or text/data) into vectors (lists of numbers) that represent their meaning. These embeddings are stored in a vector database, and RAG uses them to retrieve similar information.

->The LLM then uses the retrieved information and its own trained knowledge to generate the final answer.

Why are Embeddings Needed?

->Computers cannot directly understand human language. They understand **numbers**.

For example:

Text: "cat"
↓
Tokenizer
↓
Token: ["cat"]
↓
Embedding Model
↓

Embedding (Vector):
[0.82, -0.15, 0.44, 0.91, ...]

->This vector is a mathematical representation of the meaning of the word **"cat."**

How do Embeddings help AI?

Suppose we have these words:

Word	Embedding (simplified)
Cat	[0.82, -0.15, 0.44]
Kitten	[0.81, -0.14, 0.45]
Dog	[0.79, -0.12, 0.48]
Airplane	[-0.45, 0.91, -0.22]

Notice:

- **Cat** and **Kitten** have very similar embeddings.
- **Cat** and **Dog** are also somewhat close because they are both animals.
- **Cat** and **Airplane** are far apart.

->This is how AI understands **similar meanings**, not just exact words.

Feature Store

->A Feature Store is a centralized storage system that stores processed input features (data) that are useful for machine learning models.

-> Instead of preparing the same input data again and again, the features are stored once and provided to the ML model whenever a prediction is needed. >The ML model then analyzes these input features using the patterns it learned during training and predicts the output.

Your Football Example

Suppose you have a table like this:

Team	Goals	Assists	League Position	Last 5 Match Points
Team A	45	30	1	13
Team B	38	25	2	10
Team C	32	20	4	8

->These columns (**Goals, Assists, Position, Last 5 Match Points**) are the **processed input features** because they are the information the ML model will use.

->These processed features are stored in the **Feature Store**.

Now if you ask:

"Which team is most likely to win the next match?"

The process is:

Football Team Data
↓
Processed Input Features
(Goals, Assists, Position, etc.)
↓
Feature Store (stores the features)
↓
ML Model retrieves the features
↓
ML Model analyzes the patterns
↓
Prediction:
"Team A has the highest chance of winning."

Data Pipeline

-> A Data Pipeline is a tool or automated workflow that transfers data from the source to the required storage.

->While transferring the data, it can clean the data (fix errors or missing values), format the data (make all data follow the same structure), and process the data (convert it into useful input features).

->After that, it stores the processed data in a Feature Store or converts it into embeddings and stores it in a Vector Database.

Using your football example:

->Suppose three football websites provide team statistics.

Website	Data
Site 1	Team A, Goals = 45, Position = 1
Site 2	Team B, Goals = 38, Position = 2nd
Site 3	Team C, Goals = 32, Position = Fourth

->The **Data Pipeline** does the following while moving the data:

1. **Collects** data from all websites.
2. **Cleans** it (fixes missing values, removes duplicates).
3. **Formats** it (converts 2nd → 2, Fourth → 4, standardizes column names).
4. **Processes** it (for example, converts W,W,D,W,W into 13 points).
5. **Transfers and stores** the final processed data in the **Feature Store**.

Then, when you ask:

"Which team is likely to win?"

->The **ML model** reads the stored features and predicts the answer.

AI System Design

AI Pipeline

->An AI Pipeline is the complete workflow that starts when the user asks a question. It retrieves the required data from the storage system (Feature Store or Vector Database) and uses the appropriate AI model, such as a traditional ML model or a RAG + LLM system, to generate the final prediction or answer for the user.

How it works (based on your understanding)

->**Data Pipeline** works in the background. It collects data from websites, databases, files, etc., cleans it, formats it, processes it, and stores it.
->**AI Pipeline** starts only when the **user asks a question**. It takes the relevant stored data and passes it to the AI model to generate the output.

Your Football Example

Suppose you build a **Football Winner Predictor AI**.

Step 1: Data Pipeline (Background Work)

Football websites provide data like:

- Goals scored.
- Assists.
- League position.
- Last 5 match results.

The **Data Pipeline**:

1. Collects these statistics from football websites.
2. Cleans and formats the data.
3. Processes it into useful features.
4. Stores the processed data in the **Feature Store**.

Step 2: AI Pipeline (When User Asks)

Now the user asks:

"Which team is most likely to win the next match?"

The **AI Pipeline**:

1. Receives the user's question.
2. Retrieves the relevant football statistics from the **Feature Store**.
3. Sends those statistics to the **Traditional ML Model**.
4. The ML model analyzes the stored data using the patterns it learned during training.
5. It predicts the result and returns the answer.

Football Websites
↓
Data Pipeline
(Collect → Clean → Format → Process)
↓
Feature Store

(Stores processed football statistics)
↓
User: "Which team will win?"
↓
AI Pipeline
(Get data from Feature Store)
↓
Traditional ML Model
↓
Prediction:
"Team A has the highest chance of winning."

API

-> An API (Application Programming Interface) is a communication interface that allows different software applications or AI components to exchange data and work together.

Example

You
↓
Gmail App (the application you use)
↓
Gmail API / Communication Interface
↓
Gmail Servers
↓
Recipient's Mail Server
↓
Your Friend's Gmail App
↓
Your Friend

Based on your understanding:

- **Gmail App** → The application you use to compose and read emails.

- **Gmail API** → The communication bridge that carries the email request between applications and servers.
- **Gmail Servers** → Store, process, and route the email to the correct destination.

So when you click **Send**, your email is handed to Gmail's communication interface (API), which passes it to the Gmail servers, and the servers deliver it to your friend's email account.

Database

-> A Database is a permanent storage system that stores organized data so that AI applications and other software can retrieve and use it whenever required.

Your Football Example ⚽

Suppose you are building a **Football Winner Predictor AI**.

1. Football websites provide team statistics such as:
    a. Goals scored.
    b. Assists.
    c. League position.
    d. Last 5 match results.
2. The **Data Pipeline**:
    a. Collects the data.
    b. Cleans, formats, and processes it.
    c. Stores it permanently in the **Database**.
3. The user asks:

"Which team is most likely to win the next match?"

4. The **AI Pipeline**:
    a. Retrieves the relevant football statistics from the **Database** (or Feature Store if it is an ML prediction).
    b. Passes the data to the **ML model**.
    c. The ML model analyzes the stored data and predicts the most likely winner.
    d. The result is returned to the user.

Football Websites
↓
Data Pipeline
(Collect → Clean → Format → Process)
↓
Database (Permanent Storage)
↓
User: "Which team will win?"
↓
AI Pipeline
↓
ML Model
↓
Prediction:
"Team A has the highest chance of winning."

Inference flow

-> Inference Flow is the sequence of steps that an AI system follows after the user asks a question.

->If it is a traditional ML application, the AI Pipeline retrieves the required processed data from the Feature Store and the ML model predicts the answer. ->If it is a RAG-based application, the AI Pipeline uses RAG to retrieve relevant information from the Vector Database, passes it to the LLM, and the LLM generates the final answer for the user.

Football Example ⚽

Case 1: Traditional ML

User: "Which team will win?"
↓
Inference Flow starts
↓
Get football statistics from Feature Store
↓
ML Model analyzes the statistics

↓
Prediction:
"Team A has the highest chance of winning."

Case 2: RAG-based Football Chatbot

User: "What are the FIFA offside rules?"
↓
Inference Flow starts
↓
RAG searches the Vector Database
↓
Relevant football rule documents are retrieved
↓
LLM reads the retrieved information
↓
Answer is generated and shown to the user

Cloud AI

->Cloud AI means the AI model runs on powerful remote servers in the cloud instead of on the user's device.

How it works

User
↓
Internet
↓
Cloud Server
(AI Model runs here)
↓
Result
↓
User

ChatGPT Example

When you ask:

"Who is the CEO of Microsoft?"

Your Laptop
↓
Internet
↓
OpenAI Servers
(ChatGPT runs here)
↓
Answer generated
↓
Your Laptop

->Your laptop is **not running ChatGPT's huge model**.

->The model runs on powerful cloud servers.

Advantages of Cloud AI

✅ Very powerful models

✅ Large storage

✅ Easy to update

✅ Can use huge datasets

Disadvantages

❌ Needs internet

❌ Slight delay (latency)

❌ Data sent to server

2. Edge AI

->Edge AI means the AI model runs directly on the user's device instead of a remote cloud server.

Examples:

- Smartphone
- Laptop
- Smart Camera
- Self-driving car

Example: Face Unlock on Phone

When you unlock your phone:

Camera

↓

AI Model on Phone

↓

Face Recognized

↓

Phone Unlocked

->No cloud server is needed.

->The AI runs directly on the phone.

Tesla Example 🚙

->A self-driving car cannot wait for the internet.

Imagine:

Camera sees obstacle

↓

Send to cloud

↓

Wait 2 seconds
↓
Get answer

The car may crash.

Instead:

Camera sees obstacle
↓
AI Model inside car
↓
Brake immediately

->This is **Edge AI**.

First Understand the Main Idea

Suppose you want to build:

- A chatbot like ChatGPT.
- A football prediction AI.
- An image recognition AI.
- A recommendation system.

You need:

- Servers.
- Databases.
- Storage.
- AI models.
- APIs.

Buying all this hardware yourself is expensive.

So companies rent these services from cloud providers.

The three biggest cloud providers are:

1. Amazon Web Services (AWS)
2. Microsoft Azure (Azure)
3. Google Cloud Platform (GCP)

1. AWS AI

->AWS is the cloud platform of Amazon.

It provides:

- Servers.
- Databases.
- Storage.
- AI services.
- Machine Learning services.

Example

Suppose you build a Football Winner Predictor AI.

Instead of buying your own servers:

Users
↓
AWS Cloud
↓
Your AI Model
↓
Prediction

AWS hosts everything.

Popular AWS AI Services

- Amazon SageMaker → Build and train ML models.
- Amazon Bedrock → Use foundation models and LLMs.
- Amazon Rekognition → Image recognition.
- Amazon Lex → Chatbots.

2. Azure AI

->Azure is the cloud platform of Microsoft Azure.

It provides:

- Servers.
- Databases.
- AI services.
- Machine Learning tools.

Example

Users
↓
Azure Cloud
↓
AI Model
↓
Answer

Many enterprises use Azure because they already use Microsoft products.

Popular Azure AI Services

- Azure AI Studio.
- Azure Machine Learning.
- Azure OpenAI Service.
- Azure Cognitive Services.

3. Google Cloud AI

->Google Cloud is the cloud platform of Google Cloud.

Google is very strong in:

- AI.
- Data Analytics.
- Machine Learning.

Example

Users

↓

Google Cloud

↓

Gemini / ML Models

↓

Answer

Popular Google AI Services

- Vertex AI.
- Gemini APIs.
- Vision AI.
- Speech-to-Text.

Research Assignment

System Architecture Diagram

-> System Architecture shows all the components of an AI system and how they interact with each other. It acts like a blueprint of the entire AI application.

Example

User
↓
API
↓
AI Pipeline
↓
Feature Store / Vector Database
↓
ML Model / LLM
↓
Answer

Technology Stack

->Technology Stack is the collection of frameworks, tools, databases, cloud platforms, and programming languages used to build an AI application.

Example (ChatGPT)

- Python
- PyTorch
- GPT Model
- APIs

- Vector Database
- RAG and Cloud Infrastructure

Data Flow

->Data Flow describes how information travels through the AI system from the user's input to the final output.

Example

User Question
↓
AI Pipeline
↓
Retrieve Information
↓
LLM Processing
↓
Inference
↓
Answer

Easy Memory Trick

Topic	Simple Meaning
System Architecture	Map of the AI system 🗺️
Technology Stack	Tools used to build it 🧰
Data Flow	How data moves inside it 🔄